Mutation Testing for LLM Eval Graders, Without an LLM Judge

Erik Hill. Every quantitative claim below was verified against this repository's test suite and a live run of the tool (§8); the artifact is released with the paper and reproduces them deterministically.

Abstract

LLM evaluation suites are trusted to tell us whether a model is right, but almost nothing tells us whether the *evaluation* is right. We port mutation testing — the software-testing discipline that measures a test suite by injecting known defects and checking which get caught — to the model+prompt+**grader** stack, and we do it *deterministically, with no LLM judge*. The obstacle is that a semantic mutation has no knowable ground truth, so most work in this space uses an LLM to decide whether a mutated output is right or wrong — reintroducing exactly the untrusted judgment the evaluation was meant to remove. Our contribution is a trade: we give up mutant *complexity* to buy mutant *provability*. Each mutation is a simple, deterministic transformation whose polarity — the mutated output is provably wrong (a **defect**) or provably still correct (an **equivalent**) — is established from the eval case's own ground truth, independent of the grader under test. An operator that cannot establish polarity for a case **declines**. This makes every reported hole a fact about the grader, never an artifact of an ambiguous mutant, and it makes the tool's central promise — *never report a hole that is not a hole* — auditable. We stress that promise with eight rounds of adversarial review; the count of false positives on well-formed suites falls to zero and stays there. Pointed at faithful ports of promptfoo's shipped deterministic assertions, the tool finds, with no model in the loop, that `contains`/`icontains` used as correctness gates are blind to negation, that schema-less `is-json` is blind to wrong values (type and content), and that `word-count` used as a correctness gate is vacuous — while confirming that a well-phrased `regex` and a byte-exact `equals` are sound.

1. Introduction

Coverage tools answer "does my test suite exercise this code?" Mutation testing answers the sharper question "would my test suite *notice* if this code were wrong?" — by mutating the code and checking whether a test fails (PIT, Stryker, mutmut). LLM evaluation has the same gap one level up: a suite of graders reports pass/fail on model outputs, and we trust the greens, but we have no coverage-analog for the graders themselves. A grader can be **blind** (it passes an output that is actually wrong — a real regression would ship green), **brittle** (it fails an output that is actually correct — a false-alarm generator that gets recalibrated until it stops catching anything), or **vacuous** (it asserts nothing and cannot fail). None of these show up as a red.

The natural move — mutation-test the graders — runs into the equivalent-mutant problem at full force: a *semantic* mutation of an output has no ground-truth label, so when a grader passes a mutant you cannot tell "the grader is blind" from "the mutant is actually still correct." The field's usual escape

is an LLM judge to label outcomes, which makes the meta-evaluation only as trustworthy as the judge — the property we were trying to establish.

We take the opposite route from the usual "throw the hardest adversarial input you can generate." We restrict to mutations whose polarity is **provable** from the case's own reference, so no judge is needed and every red is a real red.

2. The provability–complexity trade

Most adversarial/mutation work on evals maximizes mutant *complexity* (semantically rich perturbations, often model-generated) and then relies on a human or an LLM to assess the outcome. The system under test there is the *model*. Our system under test is the **grader**, and for that the question is only "did the grader classify a mutant of *known* polarity correctly?" — for which complexity is worthless and provability is everything.

Complexity and provability trade off directly. A simple mutation — blank the output, wrap valid JSON in a code fence, push the single number just outside the declared tolerance — has a polarity you can prove from the reference. The most complex mutant you can generate has an *unknown* polarity, so a grader passing it is ambiguous. Simplicity is therefore not a limitation we settled for; it is a consequence of demanding a deterministic honesty guarantee. It also makes the guarantee *auditable*: you can adversarially verify "is this simple mutant really wrong for this grader?" in a way you cannot for an opaque generated one (we exploit this in §4).

The intelligence has to go somewhere. Ours goes into **mining** which simple mutation shapes correspond to defects that have actually defeated a real check, not into generating clever ones.

3. Method

Cases and ground truth. An `EvalCase` is a grader plus a reference `GradeInput` the grader is known to pass. Because the grader passes it, the reference is by definition a correct output for the task; it is the ground truth every operator reasons against. The runner refuses to mutate a case whose baseline is not green (a mis-specified reference is a loud error, not a silent skip).

Two polarities. A **DEFECT** mutation makes the output genuinely wrong; a correct grader must reject it, and if it passes, the eval is *blind* to that defect class. An **EQUIVALENT** mutation changes the output while keeping it correct (surrounding whitespace, a trailing disclaimer after a complete answer, a code fence a fence-stripping grader tolerates); a correct grader must still accept it, and if it fails, the eval is *brittle*. Most eval tooling only asks the first question; both are real failure modes and both are first-class here.

Three operator types, to keep the report fair. A surviving DEFECT is a **blind spot** (KILL: a sound grader of this kind must catch it — fix the check), a **coverage gap** (DIAGNOSTIC: this grader family is blind to this shape *by design*, e.g. a JSON validator that checks key presence not value type — add a

check, don't blame this one), or **vacuous** (SANITY: a blank/garbage floor probe the grader passes — it asserts nothing).

The operator contract — recompute or decline. An operator returns a mutant *only where it can establish that mutant's polarity for the case in hand*, and returns `None` (excluded from the score) everywhere else. Crucially, where the bar that decides correctness lives inside the grader's closure — a numeric tolerance, a grounding threshold, a trajectory-coverage threshold, whether whitespace is cosmetic for this task — the operator does **not** guess it from a module default and does **not** trust a label the grader reports about itself. The suite *declares* that bar on the case, or the operator declines. A defect operator never establishes its hole from the grader's verdict on its own mutant (that verdict *is* the hole); sound cross-checks compare two *independent* representatives.

Mined, not authored. Every operator names the concrete, documented failure it reproduces, and that provenance is enforced as a test. An *authored* mutation only tests what its author already imagined a check might miss — precisely the blind spot one is hunting. The catalog is mined from a real corpus of eval failures (a `contains` proof-gate that its own block-message satisfied; a property test that ran with `tries=1`; a JSON validator's documented key-presence-only scope; longitudinal truncation-vs-wrong-answer conflation).

4. The honesty guarantee, and evidence for it

The whole tool rests on one rule: **it never infers a hole from a verdict flip; it infers a hole from (output-proven-wrong AND grader-passed), where wrongness is established against the case's own ground truth, independent of the grader.** A false hole — a still-correct mutant reported as a blind spot, a correct grader reported brittle, a discriminating grader reported vacuous — is the one unforgivable bug, because the tool's entire value is that a red is a real red.

We do not merely assert this property; we attacked it. Over **eight rounds of adversarial cold review** (independent agents instructed to find any hole-that-is-not-a-hole, each finding reproduced by a runnable script and independently triaged against source), we drove the false-positive count down and fixed every confirmed instance, pinning each with a regression test. The early rounds surfaced the bulk of the defects — on the order of a dozen confirmed false holes at the peak. By round six the count of tool-fault false positives on a *well-formed* suite — one whose declarations are consistent with its graders — had reached **zero**, and it held through rounds seven and eight, which surfaced only *gate-symmetry* issues that bite a deliberately-misdeclared case, never a well-formed one. The regression suite now pins **ninety** such tests, one per reviewed finding class.

Every false positive had the same root cause: an operator asserting a mutant's polarity *without recomputing the graded property against the grader's real acceptance condition* — a hardcoded threshold, a trusted `grader_id` label a composite grader could carry while enforcing more, an unchecked declared tolerance, a fixed-magnitude mutant that tripped an orthogonal length constraint, a `%g`-formatted probe the grader's own tokenizer re-read as a different number, a Unicode case-fold that `swapcase` silently did not preserve, a decline-gate one operator carried and its siblings did not. The residual

work by the final rounds was making the recompute-or-decline discipline *uniform* across all eighteen operators. Two proposed fixes during review were **rejected as circular** — "run the grader on the mutant and decline if it accepts" would erase every real blind spot — a hazard the two-independent-representatives rule exists to avoid.

We regard this self-audit as a first-class result, not a footnote: it is the method demonstrating its own thesis. The tool that finds evals which quietly pass wrong answers kept catching *itself* quietly passing wrong answers, and each catch is now a test. That is only possible because the mutants are simple enough to reason about — the auditability the trade in §2 buys.

5. Findings on a deployed suite: promptfoo

To show the tool finds real holes in graders people trust — not just in its own — we pointed it at faithful ports of promptfoo's shipped **deterministic** assertions (promptfoo is among the most widely used LLM-eval frameworks; its non-LLM assertions are exactly our niche). Each port reproduces promptfoo's *documented* rule, with every divergence from the analogous in-house grader annotated so no finding is an artifact of our code; declarations are consistent with each assertion's real contract. On six realistic checks, deterministically and with no model in the loop (each finding independently re-verified against promptfoo's semantics):

- **`contains` / `icontains` used as correctness gates are blind to negation.** A "was it approved?" check with `contains: approved` passes *"I did NOT do approved; the step approved was skipped."* — a substring check cannot see polarity. Verified specifically blind: it rejects a genuinely keyword-absent reply, so it is not merely vacuous.
- **Schema-less / keys-only `is-json` is blind to wrong values (type *and* value).** A required field returned with the wrong type (`3` → `"3"`) or a wrong value of the right type (`{"approved": true}` → `false`) still parses and still has its key, so the assertion passes it. Two coverage gaps, not broken checks — only a schema with typed, valued `properties` closes them, and a `required`-only schema (a common shortcut) does not.
- **`word-count` used as a correctness gate is vacuous.** It passes a blank output and gibberish alike; it asserts only length, nothing about the answer.
- **Contrast (the fair half):** a `regex: "was approved"` on the same task *caught* the negation, and a byte-exact `equals` *caught* the blank/garbage probes and was not falsely flagged brittle. The tool named the weak checks broken and the strong ones sound.

The value is not "promptfoo is buggy" — `contains` *should* be a substring test. The value is that using a weak assertion as a correctness gate silently inherits a blind spot, and the tool surfaces exactly which, deterministically.

6. Related work

Code mutation testing (PIT, Stryker, mutmut) is the direct ancestor; it never needed an LLM because a mutant kills or survives against a deterministic test. Our claim is that this discipline *transfers to eval graders LLM-free* if one accepts provable-polarity mutants. **Metamorphic testing and CheckList**-style behavioral testing use invariances — our EQUIVALENT direction only — and target the model; we add the DEFECT direction and target the grader. **LLM-as-judge meta-evaluation** labels outcomes with a model; our whole point is to avoid that at the measurement layer.

7. Limitations

- **Shallow failure modes.** Simple provable mutants probe syntactic/structural blind spots; a grader blind to a semantically subtle wrong answer that no simple fixed edit produces will not be caught. This is the price of the trade in §2, stated plainly.
- **Grader-family coupling.** Some operators key behavior off a known grader-contract vocabulary. An external grader that does not carry a gradecore id declares its contract family on the case (`grader_family="valid_json"`) and keeps reporting its own honest id; the declaration is for gating only. This lets the tool run against a framework's graders unmodified (used for promptfoo), though it still assumes the external grader's contract maps onto a known family.
- **Mined-corpus dependence.** The catalog's value rests on the operators being mined from real defects; expanding it responsibly means mining, not authoring.

8. Reproducibility and status

The engine, the **eighteen** mined operators, a regression suite of **ninety** tests pinning every reviewed false positive, the dogfood against its own dependency's graders (score 91.4%, three honest holes — one blind spot and two coverage gaps — zero false findings), and the promptfoo experiment (six checks; two blind spots, two vacuous, two coverage gaps, zero false findings, see `external/FINDINGS.md`) are in this repository and run deterministically with no network and no model. Every number in this paper reproduces from `python -m pytest` and `evalmut run` on the tagged release. A software-testing venue (the code-mutation→evals bridge) is a natural fit; an evaluation/testing workshop is another.